



COMP 2067

Introduction to

Formal Reasoning

Lecture 8 Boolean

Author: Prof. Thorsten Altenkirch

Adapted by: Dr. Kweh Yeah Lun



University of
Nottingham

UK | CHINA | MALAYSIA

Learning Outcomes

At the end of this lecture, you will be able to

- explain the inductive type, Boolean, in Lean
- the definition of bnot, band and bor in Lean
- proving basic properties of Boolean by applying tactics: change, trivial, contradiction, refl and dsimp



Introduction

- In Lean, **bool** is defined as an **inductive type**.

[Try it!](#)

```
inductive bool : Type
| ff : bool
| tt : bool
```

- This declaration means:
 - There is a new type **bool : Type**,
 - There are two elements **tt ff : bool**,
 - These are the only elements of **bool**,
 - **tt** and **ff** are different elements of **bool**.
- **Inductive** is quite versatile, because it can be used **to define other finite types, infinite types like $\mathbb{N}$ and type constructors like list**.



Function on bool

- Let's define negation on Booleans.
- Function **bnot : bool → bool**.
- Input type: **bool**
- It will return an element of the output type: **bool**.

```
def bnot : bool → bool
| tt := ff
| ff := tt
```

[Try it!](#)



- To define a function with two inputs, like `and` for `boolean`, `band`:

```
def band : bool → bool → bool
| tt b := b
| ff b := ff
```

Try it!

- If the first argument is `tt`, that is `band tt : bool → bool`, then this is just the `identity on the 2nd argument`, because `band tt tt = tt` and `band tt ff = ff`.
- If the first argument is `ff` then the outcome will be `ff` regardless of what the 2nd argument is.
- In another words, `band ff : bool → bool` is the `constant function` which will always return `ff`.



- Next, **or** for **boolean**, **b**

Try it!

```
def bor : bool → bool → bool
| tt b := tt
| ff b := b
```

- If the first argument is **tt**, **bor tt : bool → bool** is always **tt**
- If the first argument is **ff**, **bor ff : bool → bool** is just an **identity**.



- standard infix notation for operations on bool:

$x \ \&\& \ y := \text{band } x \ y$

$x \ || \ y := \text{bor } x \ y$

- Can evaluate boolean expressions using `#reduce`:

```
#reduce ff && (tt || ff)
#reduce tt && (tt || ff)
```

[Try it!](#)



Proving some basic properties

- To reason about **bool**, use **cases x** to analyze a variable **x : bool** which means that there are two possibilities **tt** and **ff**.
- **Example:** state and prove in predicate logic that every element of **bool** is either **tt** or **ff**:

```
example :  $\forall x : \text{bool}, x = \text{tt} \vee x = \text{ff} :=$   
begin  
  assume x,  
  cases x,  
  right,  
  refl,  
  left,  
  refl,  
end
```

[Try it!](#)



- Next, let's prove `tt` is not equals to `ff`.
- First, define a predicate `is_tt : bool → Prop`
- Then, use `is_tt` to show that `tt` and `ff` cannot be equal:

```
def is_tt : bool → Prop
| ff := false
| tt := true
```

```
theorem cons : tt ≠ ff :=
begin
  assume h,
  change is_tt ff,
  rewrite ← h,
  trivial,
end
```

Try it!



- $tt \neq ff$ which is just a shorthand for $\neg (tt = ff)$, and which again is just a shorthand for $tt = ff \rightarrow \text{false}$.
- Tactic: **change**
 - **modify the current goal by replacing it with a new goal that is equivalent to the original one.**
 - rewrite the current goal into a different form that might be easier to prove or reason about.
 - particularly useful when manipulating the goal by introducing or substituting certain terms, expressions, or definitions without changing its overall meaning.



- In this example, **change** which replaces the current goal with another one that is definitely the same.

```
h : tt = ff  
⊢ false
```

- We know that **is_tt ff = false**, hence we can say **change is_tt ff** to replace the goal.
- Next, **rewrite ← h** to change the goal to **is_tt tt** which is equal to **true** and hence provable by **trivial**.
- Tactic: **rewrite**
 - **rewrite occurrences of a given expression in the current goal or hypothesis using an equality or a lemma**



- Tactic: **trivial**
 - **automatically solve goals that are considered trivially true.**
 - when applying the **trivial** tactic to a goal in Lean, it **attempts to prove the goal by applying basic simplification steps or by using built-in simplification rules.**
 - particularly useful in situations where the goal can be easily discharged without the need for complex reasoning or additional lemmas.
 - if you have a goal that involves basic arithmetic, logical operations, or properties of certain functions, **trivial** can often solve it automatically.



- Example:

```
example : tt ≠ ff :=  
begin  
  assume h,  
  contradiction,  
end
```

Try it!

- **contradiction** will solve the current goal because **there is an inconsistent assumption**.
- In this case, assuming that two different constructors of an **inductive** type are equal.



- Tactic: **contradiction**
 - **automatically close a goal by deriving a contradiction from the hypotheses.**
 - If the current set of hypotheses leads to a logical inconsistency or contradicts a known fact, the **contradiction** tactic can be applied to automatically resolve the goal.



Proving equations about bool

- Example: distributivity

```
theorem distr_b : ∀ x y z : bool,  
  x && (y || z) = x && y || x && z :=  
begin  
  assume x y z,  
  cases x,  
  dsimp [band],  
  dsimp [bor],  
  refl,  
  dsimp [band],  
  refl,  
end
```

[Try it!](#)



- After `assume x y z`, we are in the following state:

```
x y z : bool
⊢ x && (y || z) = x && y || x && z
```

- We do `case` analysis on `x` which can be either `tt` or `ff`.
- After `cases x` we are left with two subgoals

```
2 goals
case bool.ff
y z : bool
⊢ ff && (y || z) = ff && y || ff && z

case bool.tt
y z : bool
⊢ tt && (y || z) = tt && y || tt && z
```



- Let's look at the first one.
- Instruct Lean to use the definition of `&&` (i.e. `band`) by saying `dsimp [band]`:

```
y z : bool
⊢ ff = ff || ff
```

- Now apply the definition of `||` (i.e. `bor`) using `dsimp [bor]`:

```
y z : bool
⊢ ff = ff
```

- and use `refl` to dispose of this goal.



- Tactic: **refl**
 - **close a goal where the goal is an equality statement that can be proved by reflexivity.**
 - if the goal is an expression of the form $a = a$ for some term a , the **refl** tactic can be applied to automatically prove it.



- Can apply several definitions in one tactic, i.e. in the first case `dsimp [band,bor]` would have done the same.
- But in this proof, actually no need for `dsimp` at all because `refl` will automatically reduce its arguments, hence:

```
theorem distr_b : ∀ x y z : bool,  
  x && (y || z) = x && y || x && z :=  
begin  
  assume x y z,  
  cases x,  
  refl,  
  refl,  
end
```

Try it!



- Tactic: **dsimp**
 - stands for "deep simplification."
 - **to simplify expressions involving definitions, etc.**
 - **dsimp** is used in a proof state where there is an expression involving definitions to be simplified.
 - by applying **dsimp**, Lean will recursively replace the defined terms with their definitions until it can't simplify anymore.



- Based on the previous example, how about using another variable than x .
- Let's see what about z ?
- Example:

```
example :  $\forall$  x y z : bool,  
  x && (y || z) = x && y || x && z :=  
begin  
  assume x y z,  
  cases z,  
  sorry  
end
```

[Try it!](#)



- After `cases zz`: **stuck** in the following state:

```
x y : bool
⊢ x && (y || ff) = x && y || x && ff
```

- No reduction is possible because the functions are defined matching on the first argument.
- Using `cases y` would have done a bit reduction but not enough.
- `cases x` was the right choice.



Relating bool and Prop

- Example:

```
theorem and_thm :  $\forall$  x y : bool, is_tt x  $\wedge$  is_tt y  $\leftrightarrow$  is_tt (x && y) :=  
begin  
  assume x y,  
  constructor,  
  assume h,  
  cases h with xtt ytt,  
  cases x,  
  cases xtt,  
  cases y,  
  cases ytt,  
  constructor,  
  assume h,  
  cases x,  
  cases h,  
  cases y,  
  cases h,  
  constructor,  
  constructor,  
  constructor,  
end
```

Try it!



- Note that **cases x** is used for two things:
 - doing **case analysis on booleans**
 - **eliminate inconsistent assumptions.**



- Give yourself a big clap.
- Survived once again? Great! ◀◀
- Any question?



University of
Nottingham

UK | CHINA | MALAYSIA

Next lecture

- Natural numbers



“We always overestimate the change that will occur in the next two years and underestimate the change that will occur in the next ten. Don't let yourself be lulled into inaction.”

– Bill Gates

Hope you all enjoy this lecture!